

信息与软件工程学院

编译技术

主讲教师：陈安龙

第6章 运行时刻环境

- 运行时刻环境简介
- 参数传递的几种方式
- 活动记录的组织形式
- 符号表的组织形式

运行时刻环境

运行时环境（Runtime Environment）指的是程序在执行时所需的所有信息和结构。包括程序执行时的存储管理、数据结构、控制流机制、符号表的管理等多个方面。了解运行时环境是理解程序如何在计算机上运行的关键。

函数调用的管理

在编译技术中，函数调用运行时环境管理涉及到：在程序执行期间如何正确管理函数的调用、参数传递、局部变量的存储、返回值的处理，以及与控制流程相关的信息管理。良好的函数调用管理能够提高程序的可用性和性能。

函数参数传递方式

在编译技术中，函数参数传递方式它决定了参数在函数调用中的处理方式和效率。不同的参数传递机制会影响程序的性能、可读性以及维护性。主要的参数传递方式：按值传递、按结果传递、按值-结果传递、按引用传递、按名字传递。

1. 按值传递（Call by Value）

实参的值复制到形参中，函数对形参的修改不会影响实参。

代表语言：

① C 语言使用按值传递的默认方式。

② 在 Java 中，基本数据类型（如 int, float）的参数传递是按值。

③ 在 Python 中，传递不可变对象（如元组和字符串）时是按值。

2. 按结果传递（Call by Result）

实参在函数执行前不会被初始化。函数对参数的修改会在返回时写回到实参。

现代编程语言没有直接支持按结果传递，但可以通过指针或引用来模拟。

代表语言：

① Pascal 语言允许通过特定关键字实现按结果传递。

② Ada 语言也支持按结果传递，允许在返回时给参数赋值。

C 语言用指针模拟按结果传递示例

```c
#include<stdio.h>
// 定义一个按结果传递的函数
void result_pass(int *x) {
    // 在函数内部对参数的值进行修改
    *x += 10; // 将传入的值加10
}
int main() {
    int a; // 声明一个整型变量 a，但没有初始化
    // 调用函数，传递 a 的地址
    result_pass(&a); // 这里 a 尚未被初始化，模拟按结果传递

    // 为 a 显示赋值
    a = 5; // 现在赋值 a 为 5
    result_pass(&a); // 再次传递参数的地址
    printf("Result: a = %d\n", a); // 输出 a 的值
    return0;
}
```

3. 按值-结果传递（Call by Value-Result）

在函数调用时将参数的值传递给函数；而在函数执行过程中对参数的修改会在函数返回时写回到实参。

尽管大多数主流编程语言（如 C、C++、Java 等）不直接支持按值-结果传递的机制，可以通过一些方式模拟这一行为。在 Ada 语言中，可以使用按值-结果的机制进行参数传递。

以下是一个用 C++ 模拟按值-结果传递的示例。

```cpp
#include<iostream>
using namespace std;
// 定义一个结构体，模拟按值-结果传递
struct ValueResult {
    int value; // 存储值
};
// 函数模拟按值-结果传递
void modify(ValueResult x, ValueResult &result) {
    x.value += 10;  // 形参 x 增加 10
    result.value = x.value; // 将变化的值返回到 result
}
int main() {
    ValueResult a; // 定义一个结构体实例 a
    ValueResult result; // 定义一个结构体实例 result
    a.value = 5; // 初始化 a 的值为 5
    cout << "Before function call: a.value = " << a.value << endl;
    modify(a, result); //  // 调用函数，模拟按值-结果传递；a 通过值传递，result 通过引用传递
    // 修改后的值
    cout << "After function call: a.value = " << a.value << ",
           result.value = " <<result.value << endl;
    return0;
}
```

输出结果：

Before function call: a.value = 5
After function call: a.value = 5, result.value = 15

4. 按引用传递（Call by Reference）

实参地址被传递给函数，函数可以直接修改实参的值。

代表语言：

① C++：通过引用（&）符号允许按引用传递。

② C：通过指针来模拟按引用传递。

③ Rust：使用引用（&）和可变引用（&mut）来实现按引用传递。

④ Java：所有对象类型（如数组、类）都按引用传递。

按引用传递的 C++ 示例

```cpp
#include<iostream>
using namespace std;
// 定义一个按引用传递参数的函数
void modifyValues(int &x, int &y) {
    x += 10; // 修改 x，使其增加 10
    y += 20; // 修改 y，使其增加 20
}
int main() {
    int a = 5;  // 初始化 a
    int b = 10; // 初始化 b
    cout << "Before function call: a = " << a << ", b = " << b << endl;
    // 按引用传递 a 和 b
    modifyValues(a, b);
    // 输出修改后的值
    cout << "After function call: a = " << a << ", b = " << b << endl;
    return0;
}
```

输出结果：

Before function call: a = 5, b = 10

After function call: a = 15, b = 30

5. 按名字传递（Call by Name）

实际参数在函数调用时不会立即计算，而是在函数内部访问时计算。

代表语言：

① Algol：Algol 语言引入了按名字传递机制。

② Scala：Scala 支持通过惰性求值（lazy evaluation）来实现类似于按名字传递的特性。

假设下列程序按名字传递，分析其结果示例

```
program CallByNameExample;
var
    a, b: integer;
procedure P(x, y, z);  // x, y, z 将作为名字传递
begin
    y := y + 1;       // 修改 y（实际上是修改 a），y 引用 a
    z := z + x;       // z 引用 b 的值，此时 x 被计算为 a + b
end;
begin
    a := 2;            // 初始化 a
    b := 3;            // 初始化 b
    P(a + b, a, a);    // 调用 P，按名字传递参数
    // 此时调用过程：
    // y 引用 a，所以 y 将成为 2 + 1 = 3
    // 则 z 被计算为 3（当前的 a 值） + (a + b) => 3 + (3 + 3) = 9
    write('After calling P, a = ', a, ', b = ', b); // 输出 a 和 b 的值
end.
```

最终输出为： After calling P, a = 9, b = 3。

存储分配的典型方式

1. 静态存储分配（Static Memory Allocation）

   ①静态存储分配是指在编译时为变量分配固定的存储空间；这种方式适用于全局变量和静态变量。

   ②整个程序执行期间存在，存储区在程序加载时分配，并在程序结束时释放。

   ③内存大小固定，分配简单有效，但灵活性较差。

   ④访问速度快，因为内存地址在编译时已知。

存储分配的典型方式

2. 动态存储分配（Dynamic Memory Allocation）

①动态存储分配允许在程序运行时根据需要分配和释放内存；动态内存通常位于堆中。

②程序运行期间，直到程序明确释放内存；灵活性高，适用于不知道大小的结构（如链表、树等）。

③管理更复杂，需要手动分配和释放，可能会引发内存泄漏。

④通常比静态分配慢，由于寻址开销。

存储分配的典型方式

3. 栈式分配（Stack Allocation）

①在函数调用过程中为局部变量和函数参数分配内存的一种机制。栈式分配利用了系统的调用栈（Call Stack）来动态管理内存，具有高效的内存使用特性。

②随着函数调用和返回，活动记录(栈帧)自动分配和释放。相对简单且快速。

③适合小型和短生命周期的存储需求，内存使用效率高。

④因为遵循后进先出原则 (LIFO)，所以访问速度快。。

函数的活动

函数的一次执行称为函数的一次活动，函数的活动需要：

① 可执行代码

② 存放所需信息的数据结构：活动记录

活动树（Activation Tree）

- 用于描述程序在运行过程中函数调用关系的结构。它通过树形结构展现了函数的调用层次和执行顺序，以及各个函数调用的上下文信息。这种模型对于分析和优化程序的行为，特别是涉及到递归和函数嵌套调用时尤为重要。

活动树（Activation Tree）

- 活动树呈现了一个程序在运行时的动态调用结构，通常由以下几个要素构成：

① 节点（Node）：每个节点代表一个函数调用，节点包含函数的名称和相关信息，如参数、局部变量等。

② 边（Edge）：边表示函数调用之间的关系，树中父节点是调用方，子节点是被调用的函数。

③ 高度和深度：树的高度反映了函数调用的最深层次，深度表示从根节点到当前节点的路径长度。

活动树的例子

```
int a[11];
void readArray() { /* Reads 9 integers into a[1], ..., a[9]. */
    int i;
    ...
}
int partition(int m, int n) {
    /* Picks a separator value v, and partitions a[m..n] so that
       a[m..p-1] are less than v, a[p] = v, and a[p+1..n] are
       equal to or greater than v. Returns p. */
    ...
}
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0] = -9999;
    a[10] = 9999;
    quicksort(1,9);
}
```

```
enter main()
    enter readArray()
    leave readArray()
    enter quicksort(1,9)
        enter partition(1,9)
        leave partition(1,9)
        enter quicksort(1,3)
            ...
        leave quicksort(1,3)
        enter quicksort(5,9)
            ...
        leave quicksort(5,9)
    leave quicksort(1,9)
leave main()
```

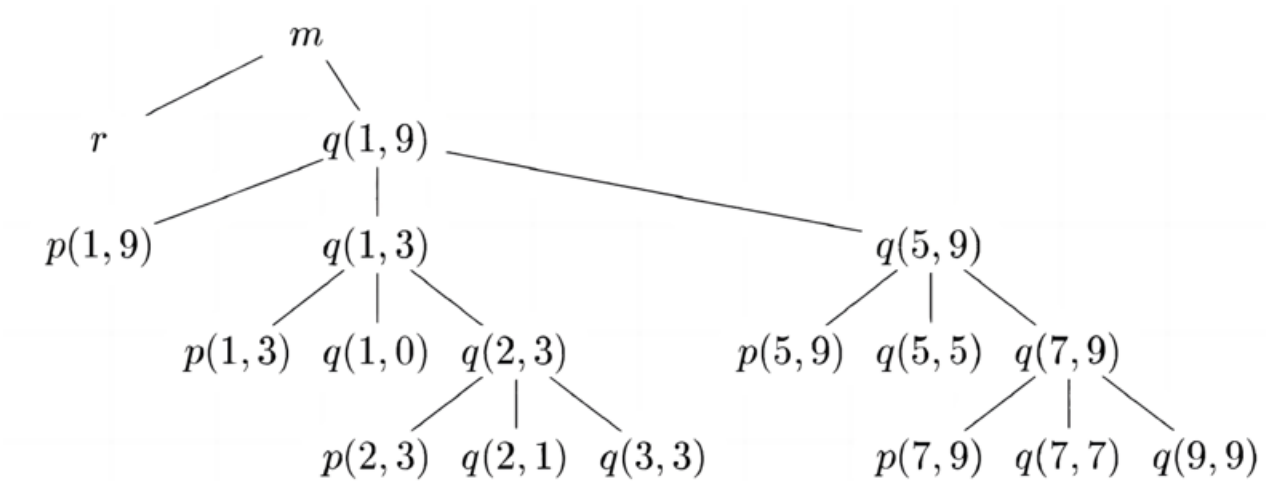
图 7-2 中程序的可能的活动序列

活动树的例子（1）

- 过程调用（返回）序列和活动树遍历对应

```
enter main()
    enter readArray()
    leave readArray()
    enter quicksort(1,9)
        enter partition(1,9)
        leave partition(1,9)
        enter quicksort(1,3)
            ...
        leave quicksort(1,3)
        enter quicksort(5,9)
            ...
        leave quicksort(5,9)
    leave quicksort(1,9)
leave main()
```

图 7-2 中程序的可能的活动序列



活动记录

在编译技术中，活动记录（Activation Record）是一种用于管理函数执行上下文的数据结构。它包含了函数调用期间所需的所有信息，例如参数、局部变量、返回地址等。活动记录的设计旨在支持函数调用和返回的机制，尤其在实现递归和多重调用时尤为重要。

活动记录的结构

① 实在参数
② 返回值
③ 控制链(可选)
④ 访问链(可选)
⑤ 保存的机器状态
⑥ 本地(局部)数据
⑦ 临时变量

① 实在参数：函数传入的参数

② 返回值：函数的返回值

③ 控制链(可选)：指向调用者活动记录的指针，用于当调用返回时，将当前栈顶正确切换到调用者的活动记录；

④ 访问链(可选)：用于在可嵌套定义的过程中指示访问非本地数据；

⑤ 保存的机器状态：如程序计数器，寄存器等；

⑥ 过程内部声明的局部数据：如 VAR X，Y :INTEGER等；

⑦ 临时变量：源程序中不出现的、由编译程序产生的变量，如表达式x+y+z求值时产生的T1，T2，...；

由控制栈来管理活动记录

- 过程调用和返回由控制栈进行管理
- 每个活跃的活动对应于栈中的一个活动记录
- 活动记录按照活动的开始时间，从栈底到栈顶排列

由控制栈来管理活动记录

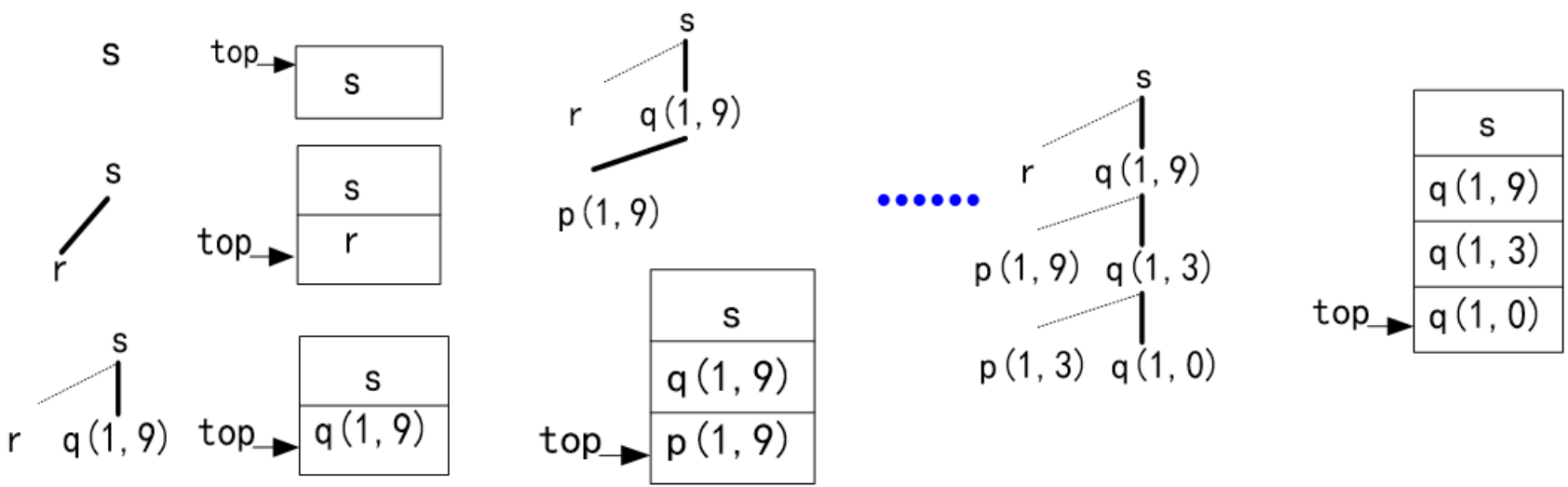
控制栈的两个重要指针(全程量)

top： 控制栈的栈顶

sp： 每个活动记录中的相对寻址，也代表此活动记录

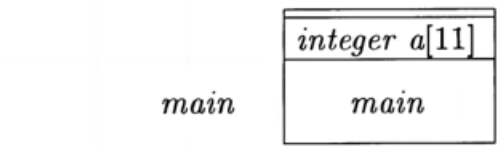
在整个的程序运行过程中，top指示栈顶，sp指示当前栈顶的活动记录。

[例] 控制栈的变化过程s—>r>—q(1,9)—>q(1,3)—>q(1,0)

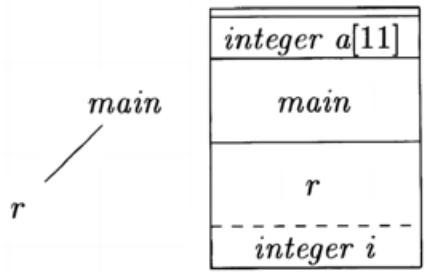


运行时刻栈的变化

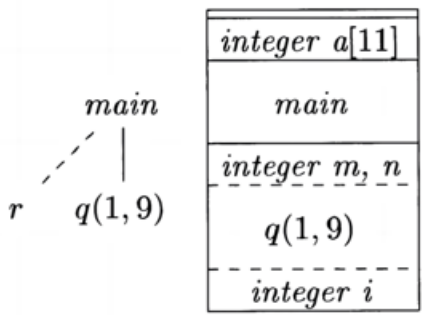
- a[11]为全局变量
- main没有局部变量
- r有局部变量i
- q的局部变量i，和参数m,n



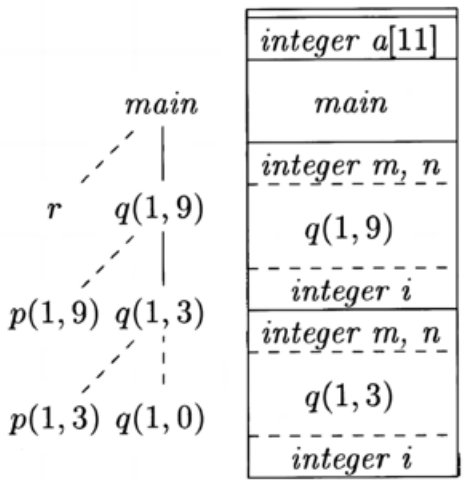
a) γ r进栈



b) γ被激活



c) γ被弹出栈 ,$q(1,9)$ 被压栈



d) 控制返回到 $q(1,3)$

调用者/被调用者的活动记录

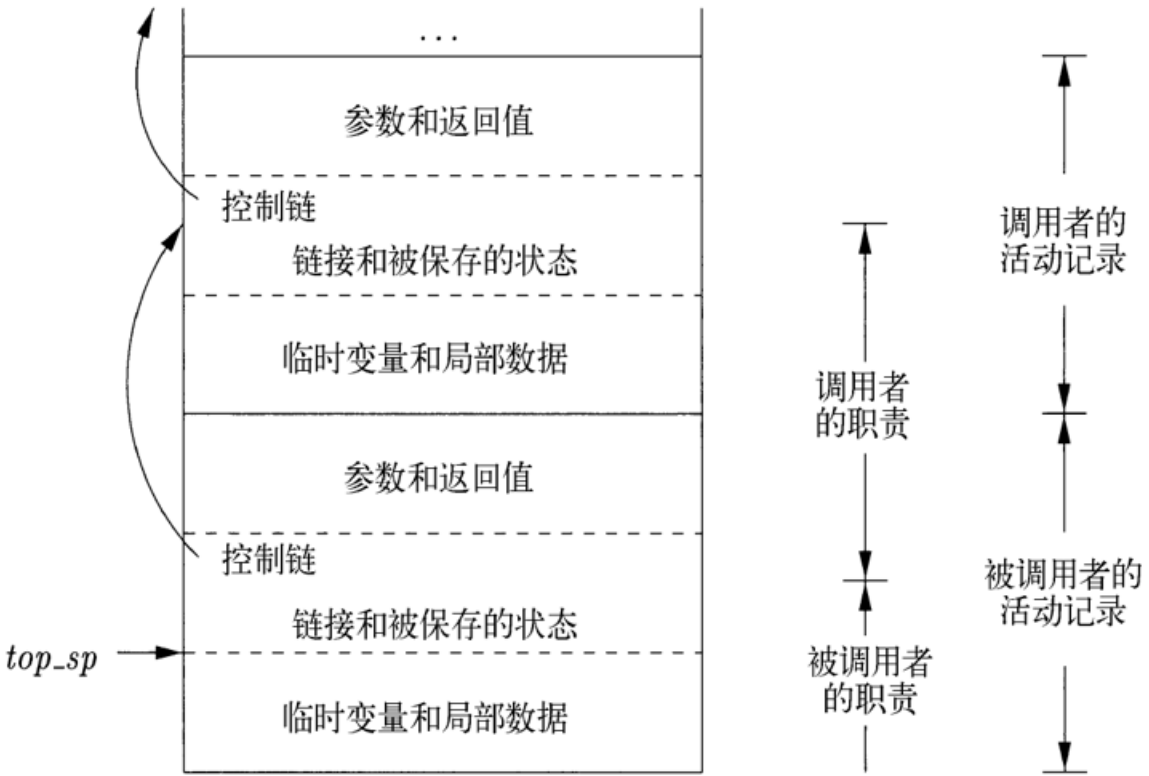


图 7-7　调用者和被调用者之间的任务划分

栈中的变长数据

- 如果数据对象的生命期局限于过程活动的生命期，就可以分配在运行时刻栈中。

  变长数组也可以放在栈中

- top指向实际的栈顶
- top_sp用于寻找顶层记录的定长字段

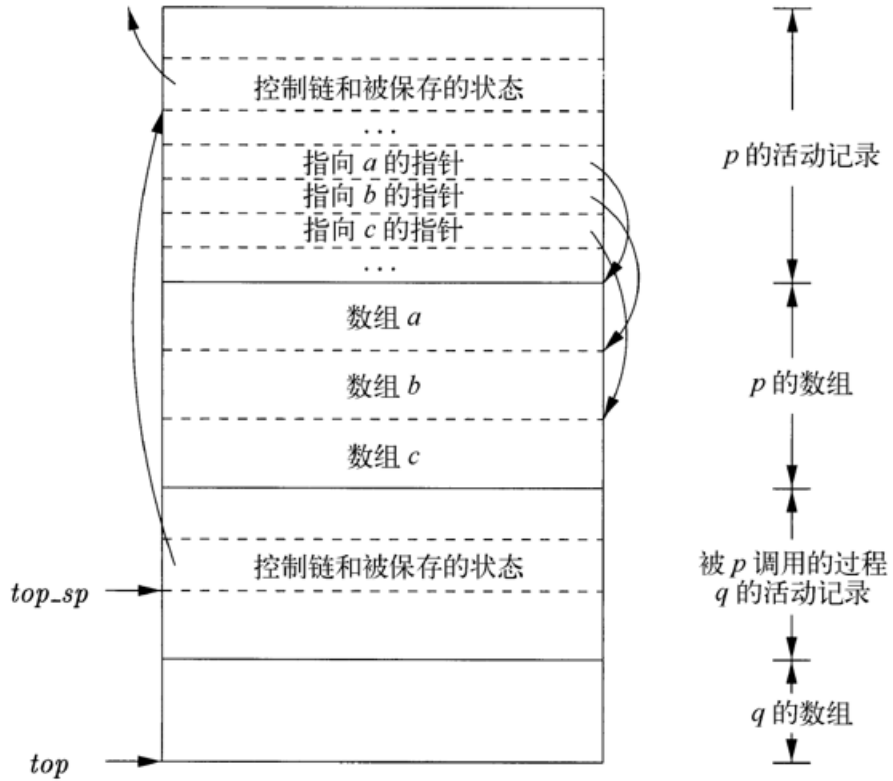


图 7-8　访问动态分配的数组

没有嵌套过程时的数据访问

- C语言中，每个函数能够访问的变量
  - ① 函数的局部变量：相对地址已知，且存放在当前活动记录内，top_sp指针加上相对地址即可访问
  - ② 全局变量：在静态区，地址在编译时刻可知
- 很容易将C语言的函数作为参数进行传递
  - ① 参数中只需包括函数代码的开始地址。
  - ② 在函数中访问非局部变量的模式很简单，不需要考虑过程是如何激活的。

嵌套过程时的数据访问

- PASCAL中，如果过程A的声明中包含了过程B的声明，那么B可以使用在A中声明的变量。
- 当B的代码运行时，如果它使用的是A中的变量。那么这个变量指向运行栈中最上层的同名变量。
- 但是，不能通过嵌套层次直接得到A的活动记录的相对位置。必须通过访问链访问

```
void A( )
{
        int     x,y;
        void    B( )
        {       int b;
                x = b+y;
        }
        void    C( ){B( );}
    C( );
    B( );
}
```

当A调用C，C又调用B时:

A的活动记录
C的活动记录
B的活动记录

当A直接调用B时:

A的活动记录
B的活动记录

嵌套深度

- 嵌套深度是正文概念，可以根据源程序静态地确定

① 不内嵌于任何其他过程中的过程，嵌套深度为1

② 嵌套在深度为i的过程中的过程，深度为i+1.

深度为1

sort

深度为2

readArray，exchange，quicksort

深度为3

partition

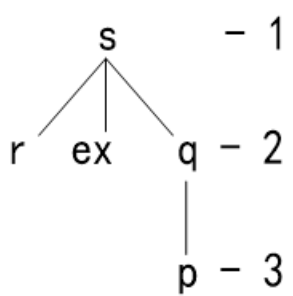
```
1) fun sort(inputFile, outputFile) =
       let
2)         val a = array(11,0);
3)         fun readArray(inputFile) = ··· ;
4)               ··· a ··· ;
5)         fun exchange(i,j) =
6)               ··· a ··· ;
7)         fun quicksort(m,n) =
               let
8)                 val v = ··· ;
9)                 fun partition(y,z) =
10)                      ··· a ··· v ··· exchange ···
               in
11)                ··· a ··· v ··· partition ··· quicksort
               end
       in
12)        ··· a ··· readArray ··· quicksort ···
       end;
```

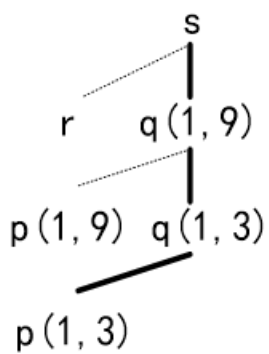
利用访问链访问非本地数据

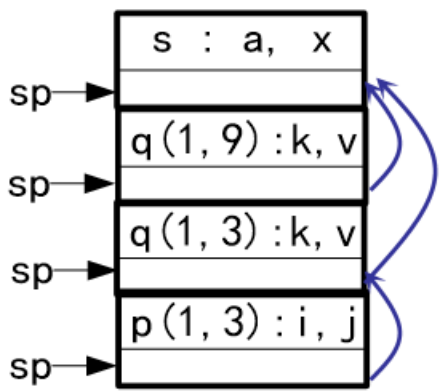
设过程p的嵌套深度是n_p，过程p中引用一个嵌套深度为na且na≤np的变量a。设p和a的层次之差为x，即$x= n_p-n_a$。则a的存储可以如下方式找到：

① 当控制在p中，p的一个活动记录肯定在栈顶。从栈顶的活动记录中追踪访问链n_p-n_a次。(n_p-n_a的值是静态作用域规则决定的，可以在编译时计算得到)；

② 追踪访问链n_p-n_a次后，找到a的声明所在过程的活动记录的访问链sp(a)。







如何在调用序列中建立访问链

- $n_p \geq n_x$的情况

  ① p和x的嵌套深度分别为1，2，…，n_x-1的外围过程肯定相同

  ② 追踪访问链$n_p - n_x + 1$次，到达了静态包围x和p的且离它们最近的那个过程的最新活动记录

  ③ 所到达的访问链就是x的活动记录中的访问链应该指向的那个访问链

如何在调用序列中建立访问链

- $n_p \geq n_x$的情况

① p和x的嵌套深度分别为1，2，…，n_x-1的外围过程肯定相同

② 所到达的访问链就是x的活动记录中的访问链应该指向的那个访问链

③ 追踪访问链$n_p - n_x + 1$次，到达了静态包围x和p的且离它们最近的那个过程的最新活动记录

访问链的示例

在前面快排序过程从s到q(1,9)、q(1,3)、p(1,3)、e(1,3)控制栈与访问链的变化过程

① s调用q(1,9)：ns-nq+1=1-2+1=0

将q(1,9)的访问链指向s的访问链地址

② q(1,9)调用q(1,3)：nq-nq+1=2-2+1=1

从q(1,9)追踪一次，到达s的访问链

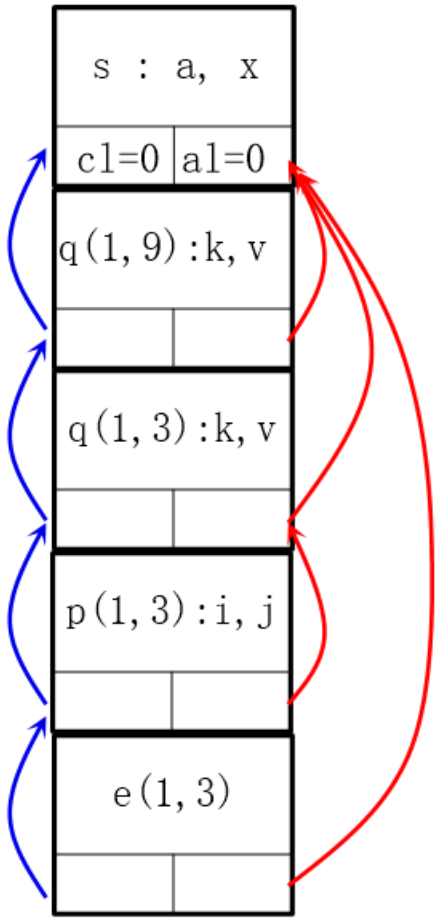
③ q(1,3)调用p(1,3)：nq-np+1=2-3+1=0 处理与①同

④ p(1,3)调用e(1,3)：np-ne+1=3-2+1=2

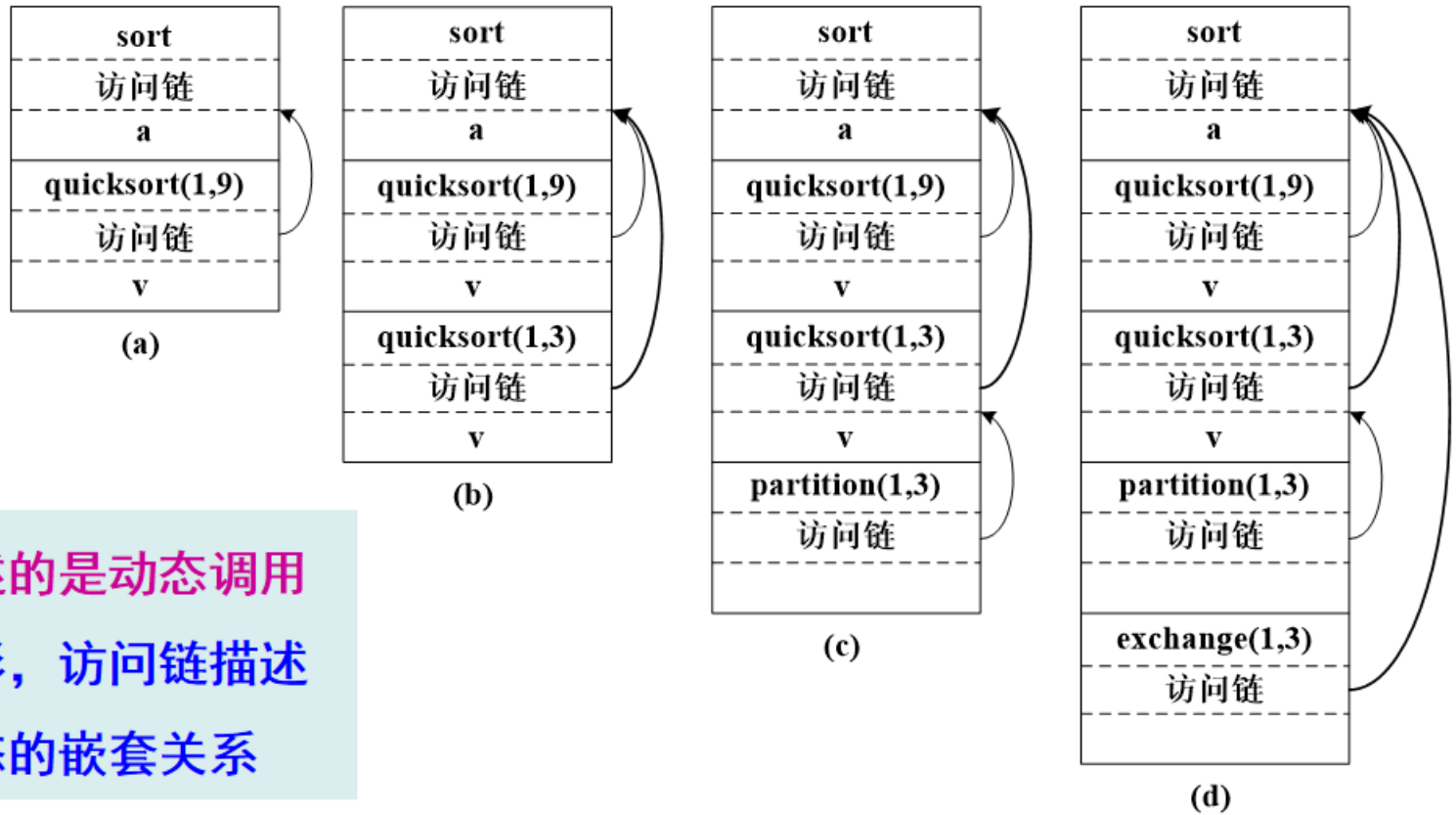
从p(1,3)追踪两次，到达s的访问链

提示：访问链反映静态层次嵌套

控制链反映动态调用嵌套



访问链的示例（续）



栈内描述的是动态调用时的情形，访问链描述的是静态的嵌套关系

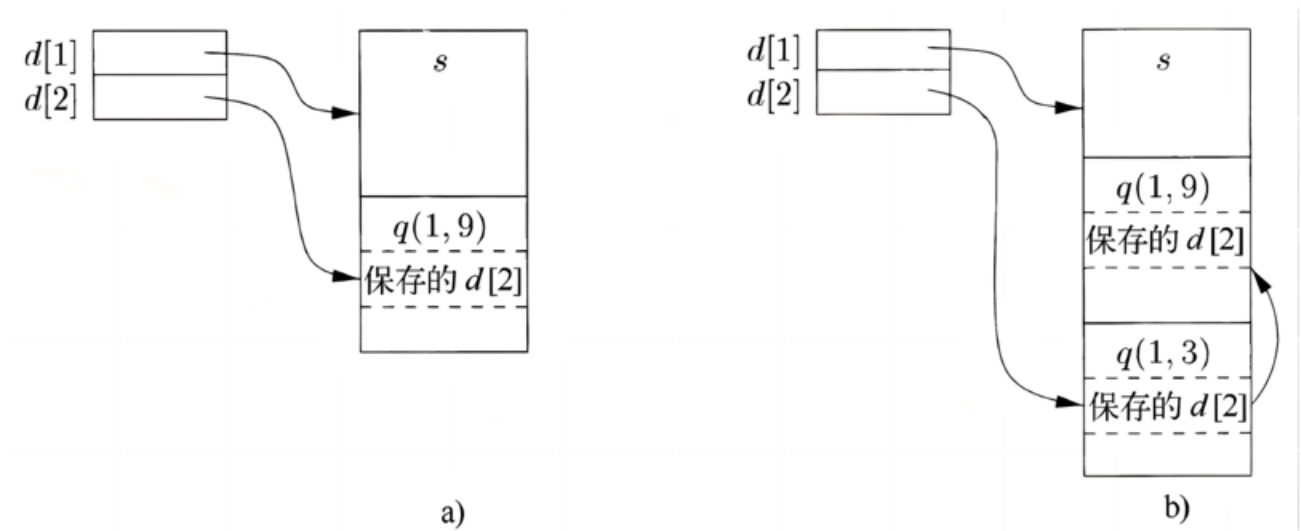
Display显示表

- 在编译技术中，Display（显示表） 是一种用于管理作用域和变量访问的一种数据结构。它特别适用于处理静态作用域的编程语言，能够在函数调用过程中提升局部和全局变量的访问效率。通过有效地使用显示表，编译器可以快速定位变量的值，尤其是在多层嵌套的作用域中。

显示表的工作机制

- **基本结构：** 通常是一个数组或链表，每个元素对应一个活动记录。每个条目通常包含指向一个活动记录的指针，这个活动记录存储了一个函数的局部变量、参数和其他相关信息。 显示表通常具有一个较低的开销，因为它通常只在函数调用和返回时更新。
- **初始化和更新：** 在程序开始时，显示表被初始化，包含全局变量的指针。 当一个函数被调用时，调用该函数的当前作用域的值被压入显示表中。 函数返回时，该条目会被弹出，恢复到先前状态。
- **变量访问：** 当一个变量在函数内部被访问时，显示表允许编译器直接查找相关的活动记录，快速获取变量的值。

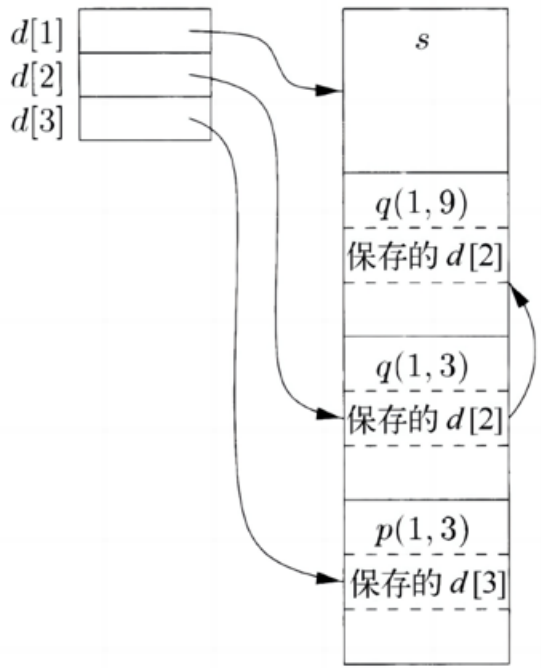
显示表的例子



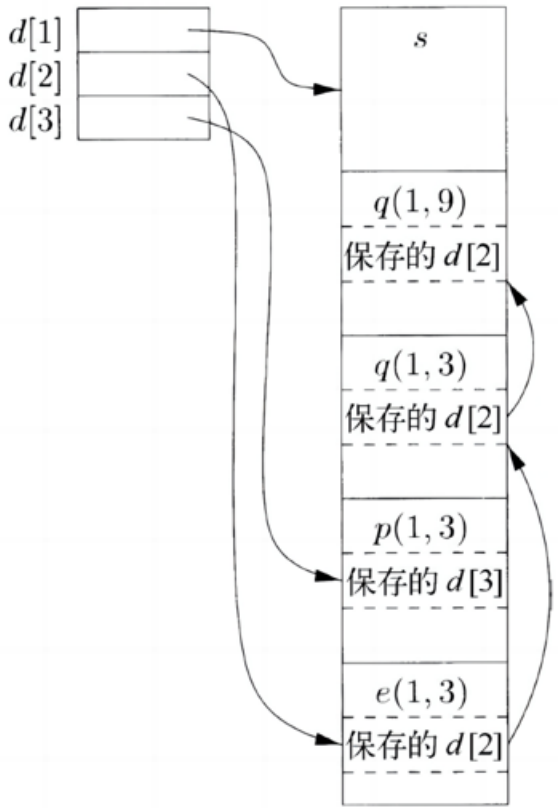
q(1,9)调用q(1,3)时，q的深度为2

显示表的例子

q(1,3)调用p，p的深度为3



c)



d)

q调用e，e的深度为2

符号表（Symbol Table）

在编译技术中，符号表（Symbol Table）是一个关键的数据结构，用于存储程序中所有标识符（如变量名、函数名、类型及其作用域、存储位置等）的信息。它支持编译器的词法分析、语法分析和语义分析等多个阶段。符号表的组织形式直接影响编译器的性能和功能。

符号表的数据结构

符号表的每一项通常称为符号表项（也称为entry），每个条目包含与标识符相关的多个信息。一个典型的符号表项结构可以如下定义：

```
typedef struct Symbol {
    char *name;              // 标识符名
    char *type;              // 数据类型（整型、浮点型、数组等）
    int scopeLevel;          // 用于标识符的作用域层级(全局、局部、参数等)
    int address;             // 地址或偏移量
    int storageClass;        // 存储类别（自动、静态、寄存器等）
    int isInitialized;       // 是否已初始化的标志
    struct Symbol *next;     // 链表指针，用于链地址法处理冲突
} Symbol;
```

符号表的组织形式

符号表可以根据不同的需求和实现方式进行组织。主要有以下几种形式：

1. 线性表

① 实现：使用链表或者简单的数组来存储符号。

② 优点：简单直观，易于实现。

③ 缺点：查找效率低，平均时间复杂度为 O(n)。不适合大型程序或复杂的符号查找。

符号表的组织形式

2. 哈希表

① 实现：哈希表通过哈希函数将标识符映射到表中的某个位置，解决哈希冲突通常采用链地址法或开放地址法。

② 优点：可以快速查找、插入和删除操作，平均时间复杂度为 O(1)。

③ 缺点：需要选择合适的哈希函数，并处理哈希冲突; 动态调整可能复杂。

符号表的组织形式

3. 树结构

① 实现：可以使用平衡二叉树（如 AVL 树、红黑树）存储符号表。

② 优点：支持快速查找（时间复杂度 O(log n)）和有序遍历。

③ 缺点：实现复杂，维护成本高。

符号表的组织形式

4. 分层符号表（Scope Table）

① 实现：每个作用域有一个独立的符号表，使用链表或树结构连接；随着程序的结构，多个作用域的符号表按照层级连接。

② 优点：支持嵌套作用域和多种复杂性，查找时能在局部作用域和全局作用域中快速切换。

③ 缺点：管理和实现复杂，需要处理符号的创建和删除。

